Static Analysis and Key Prompts

I. Static Analysis

The results of static analysis mainly include function definitions, available actions, data/control dependencies, call graphs, and error-propagation paths. These results are utilized by the EH-Database, Repair Agent, and Validate Agent.

We demonstrate the static analysis process of EH-Fixer using the following example:

```
1 static int max1111_read(struct device *dev, int channel){
      ...
2     err = spi_sync(data->spi, &data->msg);
3     if (err < 0) {
4         dev_err(dev, "spi_sync failed with %d\n", err);
5         mutex_unlock(&data->drvdata_lock);
6         return err;
7     }
     ...
8}
```

Fig. 1: Example of Buggy Function.

For a target software system, EH-Fixer first constructs its AST using Tree-sitter:

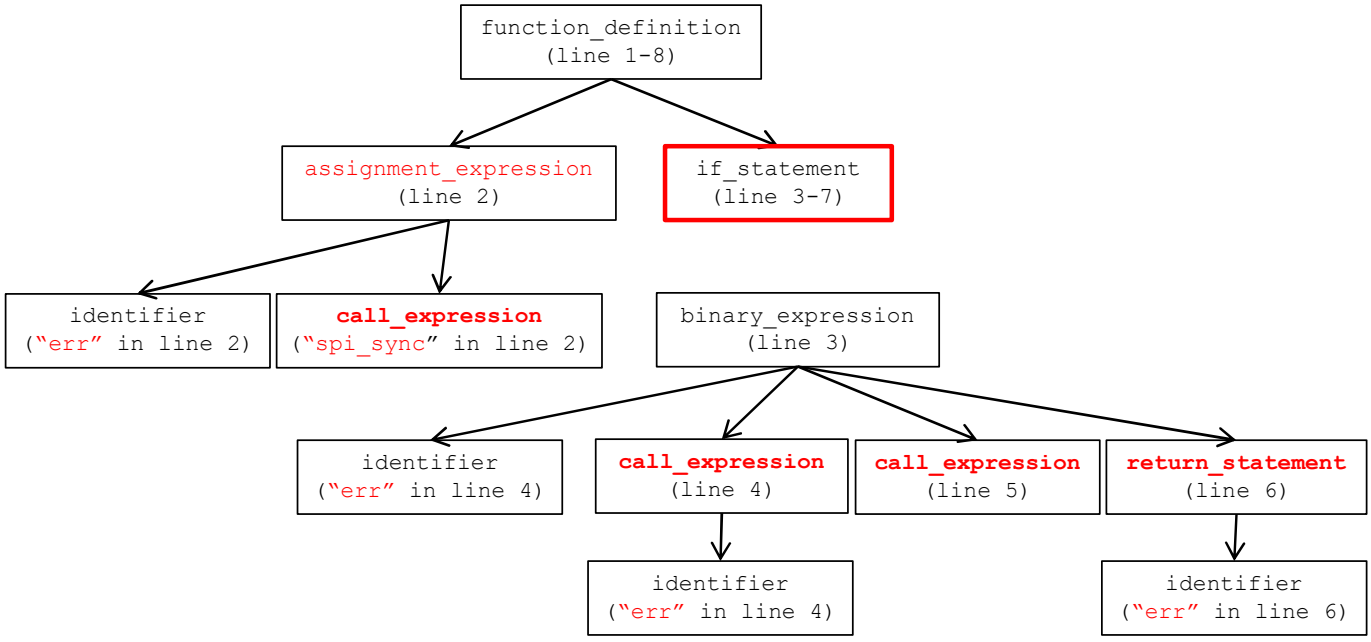

Fig. 2: Example of AST.

Upon inputting the buggy function shown in Fig. 1, the tool analyzes its AST (Fig. 2) and generates the analysis results presented in Fig. 3:

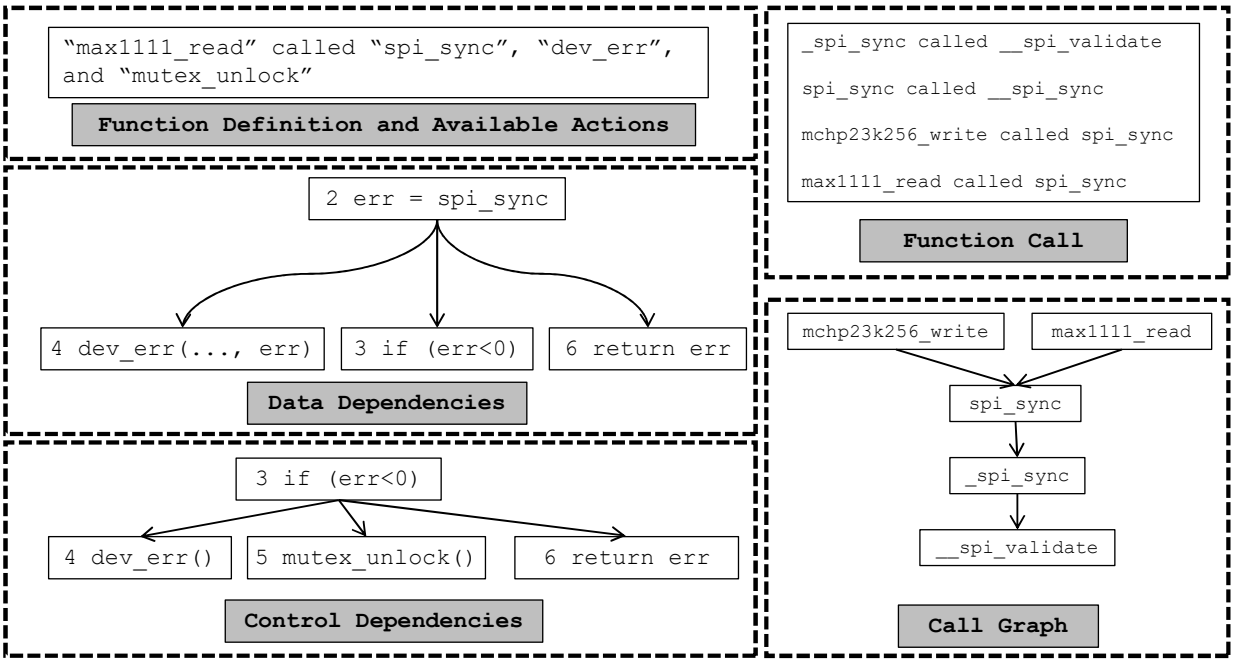

Fig. 3: Example of Static Analysis Results.

Specifically, function definitions and available actions (such as function calls and variables) are extracted by analyzing AST nodes such as `function_definition`, `call_expression`, and `identifier`. Data dependencies are identified through the analysis of assignment statements (e.g., "err" and "*spi_sync*" in line 2) and variable names (e.g., "err" in lines 2, 3, 4, and 6). Control dependencies within a function are captured by examining branching nodes (e.g., `if_statement`). Based on the identified function calls, the Call Graph is then constructed.

Following the call graph, we recursively trace the error propagation path from leaf to root nodes. As shown in Fig. 4, EH-Fixer analyzes `_spi_sync`, then `spi_sync`, and proceeds with `mchp23k256_write` and `max1111_read`. When analyzing `spi_sync`, the analysis results stored for `_spi_sync` are directly reused, preventing redundant analysis. Each function is analyzed only once, which mitigates the search space explosion problem in inter-procedural static analysis.

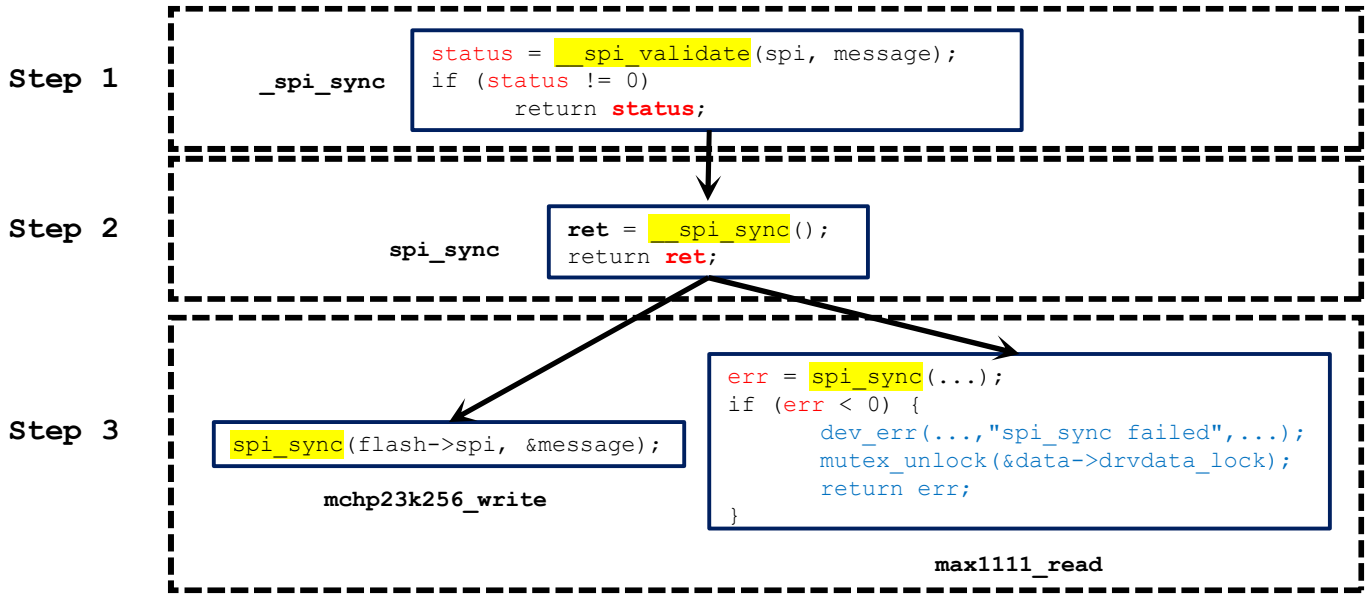

Fig. 4: Example of Propagated Path Construction.

II. KEY PROMPT

Key prompts mainly include: the error-handling strategies leaning in the EH-Database, the patch generation and patch revision in the Repair Agent, and patch validation in the Validate Agent.

Fig. 5 lists the prompts used to learn the error-handling strategies in EH-Database. Variables are enclosed in curly braces ({}) and are filled in by EH-Fixer when analyzing different code snippets. Taking the code snippet in Fig. 1 as an example, `bug location` is filled with ``line 2 of function max1111_read'', `error` with ``spi_sync(data->spi, &data->msg);'', and `buggy function` with ``max1111_read''. The `Source code of buggy function` contains `the source code of the function max1111_read`, while the `Source code of caller functions` includes `code snippets related to the error`, sliced based on data/control flow dependencies.

Instruction

Assume the role of an experienced software engineer, and ***learn error-handling strategies*** from the error handling for {bug location}. You will be provided with the source code of {buggy function} and its callers as contextual information. You should proceed according to the following steps:

1. ***Analysis error Impact***:

How {error} is used by {buggy function}, and how it may affect the functionality of {buggy function} in case of an error.

How {buggy function} is used by its callers, and how {buggy function} might affect callers when it goes wrong.

2. ***Analysis Handling Actions***:

List how {error} in the {buggy function} is handled: whether {buggy function} is stopped; what information is logged; which resources are cleaned up; which error code is returned.

3. ***Analysis Relation Pairs***:

Correlate these impacts with handling actions, forming concise relationship pairs.

Example

Output: {

"Impact": ["SPI transmission failure", "Cannot pass input data for caller functions"],

"Handling Actions": ["Logging", "Resource cleanup", "Earlystop", "Propagate error"],

"Relationship Pairs": [

<"SPI transmission failure", ["Logging", "Resource cleanup", "Early stop"]>,

<"Cannot pass input data for caller functions", ["Propagate error"]>

]

}

Constraints

1. If there are functions whose source code also needs to be analyzed but are not in the Contextual Information, output them in JSON format {**"Function Retrieval"**: [function names]}
2. Focus only on the error produced by {error}.
3. Output in JSON format as shown in the Example.
4. If there is no corresponding action, leave the list empty.
5. "return;" should be outputted as "NULL"

Contextual Information

Source Code: [buggy function and caller functions]

Additional Information: [Information retrieved based on "Function Retrieval"]

Fig. 5: Prompt of EH-Database.

Fig. 6 and Fig. 7 shows the prompts used by the Repair Agent when generating or modifying patches.

When multiple functions are fixed collectively, they are listed in square brackets ([]+). For the logging action in Line 4 of Fig. 1, `Available Actions` would include "(`Logging`, `dev_err`, `"Sliced code snippets"`)".

Instruction

Assume the role of an experienced software engineer. [{bug location} may lack error handling.]+ You will be provided with: [the source code of {buggy function} and its callers;]+ <impact, action> pairs demonstrate how other errors are handled; available actions that can be used for patch generation as contextual information. For each error, you should proceed according to the following steps:

1. ***Analysis error Impact***:

How {error} is used by {buggy function}, and how it may affect the functionality of {buggy function} in case of an error.

How {buggy function} is used by its callers, and how {buggy function} might affect callers when it goes wrong.

2. ***Predict Handling Actions***:

Refer to the <impact, action> of other errors, and whether {error} needs error handling based on its impact, and if so, what handling actions are needed.

3. ***Generate Patch***:

If error handling is required, generate a patch for {error} based on the predicted handling actions and the available actions.

Example

{bug location}:{

"Impact": ["SPI transmission failure", "Cannot pass input data for caller functions"],
"Need Handling": True,
"Relationship Pairs": [
<"SPI transmission failure", ["Logging", "Resource cleanup", "Early stop"]>,
<"Cannot pass input data for caller functions", ["Propagate error"]>
],
"Handling Actions": ["Logging", "Resource cleanup", "Earlystop", "Propagate error"],
"Patch": "+if (err < 0){
+dev_err(dev, "spi_sync failed with %d\n", err);
+mutex_unlock(&data->drvdata_lock);
+return err;}"

}

Constraints

1. Focus only on the error produced by {error}, and only use functions/return values from the Contextual Information.

2. If there are functions whose source code also needs to be analyzed but are not in the Contextual Information, output them in JSON format {**"Function Retrieval"**: [function names]}

3. If there are resources that need to be cleaned up, but their cleanup function is not in the Contextual Information, output them in JSON format {**"Cleanup Retrieval"**: [function names]}

4. Output in JSON format as shown in the Example.

Contextual Information

Source Code: [buggy function and caller functions]
Relation Pairs: [retrieved from functions in the same file or propagation path]
Available Actions: [retrieved from functions in the same file or propagation path]
Additional Information: [Information retrieved based on "Function Retrieval" and "Cleanup Retrieval"]

Fig. 6: Prompt of Repair Agent for Patch Generation.

When modifying a patch, `Incorrect Patch` refers to the full patch code, while `Validation Result` is a natural language description returned by the Validate Agent or the static check. For instance, it could include "``+if (dev) +return;'' is checking the parameters of max1111_read, not related", and "free_dev is not in the contextual information".

Instruction

Assume the role of an experienced software engineer. {bug location} is missing error handling. You will be provided with: the source code of {buggy function}; available actions that can be used for patch generation as contextual information. It requires the following handling actions: {Predicted Handling Actions}. The last patch you generated is: {Incorrect Patch}. It had the following problems: {Validation Result}. Please revise the patch.

Example

Output: {

"**Patch**": "+if (err < 0){
+dev_err(dev, "spi_sync failed with %d\n", err);
+mutex_unlock(&data->drvdata_lock);
+return err;}"

}

Constraints

Same

Contextual Information

Source Code: [buggy function]

Available Actions: [retrieved from functions in the same file or propagation path]

Additional Information: [Information retrieved based on "Function Retrieval" and "Cleanup Retrieval"]

Fig. 7: Prompt of Repair Agent for Patch Modification.

The Validate Agent uses the source code of the buggy function and predicted handling actions as context to determine whether the candidate patch generated by the Repair Agent contains irrelevant modifications. If such modifications are found, it returns a description of the issue.

Instruction

Assume the role of an experienced software engineer. {bug location} is missing error handling. You will be provided with: the source code of {buggy function} as contextual information. It requires the following handling actions: {Predicted Handling Actions}. The last patch you generated is: {Candidate Patch}. Please check whether all modifications are related to {error}.

Constraints

1. If a modification is not handling an error in {error}, it is considered irrelevant. Such as adding a check for the {buggy function} parameter.
2. Output in the following JSON format:

```
{
    "Plausible": True/False,
    "Problems": []
}
```

Contextual Information

Source Code: [buggy function]

Fig. 8: Prompt of Validate Agent.